Warp Modularity

From paper IORs to a code-level interface

Internal note

What we want

A single interface every Warp phase satisfies, that is visibly the same shape as the paper's IOR signature.

- `lib.rs::prove` reads as a chain of typed transitions.
- Each transition: $(\text{stmt}, \text{wit}, \text{or}_{\text{in}}) \mapsto (\text{stmt}', \text{or}_{\text{out}})$.
- Adding a phase: a struct + a trait impl. No bespoke plumbing.

Why Warp wasn't easy modularly

The paper IOR formalism doesn't share state across IORs. Real Warp does.

Three concrete frictions:

1. **Shared oracle f.** TwinConstraint, OOD, Batching, Proximity all consume the same codeword. A literal IOR-per-phase would rematerialise.
2. **Transcript ordering coupling.** Shift-query indices are squeezed once and consumed by both Batching and Proximity. No phase "owns" the squeeze.
3. **Statement / witness / oracle conflation.** The pre-refactor function had $\sim$10 mixed-purpose args; the paper's tripartite split was invisible.

What an IOR signature is

The Warp paper structures the protocol as a sequence of Interactive Oracle Reductions, each one having the composition rule:

$$A : (\mathrm{stmt}_A, \mathrm{wit}_A) \mapsto (\mathrm{stmt}'_A, \mathsf{O}[f])$$

$$B : (\mathrm{stmt}_B, \mathsf{O}[f]) \mapsto \mathrm{stmt}'_B$$

$$A; B : (\mathrm{stmt}_A, \mathrm{stmt}_B, \mathrm{wit}_A) \mapsto (\mathrm{stmt}'_A, \mathrm{stmt}'_B)$$

Five typed components per IOR:

	Prover	**Verifier**
`Statement`	✓	✓
`Witness`	✓	—
`InputOracles`	by data	by handle
`ReducedStatement`	✓	✓
`OutputOracles`	by data	by handle

The IOR trait we built

v1 was a thin `ProverPhase` with opaque `Output`; v2 mirrors the paper:

```
pub trait IOR {
    type Statement; type Witness;
    type ProverInputs; type VerifierInputs;
    type ReducedStatement;
    type ProverOutputs; type VerifierOutputs;

    fn prove(&self, ts: &mut ProverState, stmt: &Self::Statement,
             wit: Self::Witness, inputs: Self::ProverInputs)
        -> Result<(Self::ReducedStatement, Self::ProverOutputs), ProverError>;

    fn verify<'a>(&self, ts: &mut VerifierState<'a>,
                  stmt: &Self::Statement, inputs: Self::VerifierInputs)
        -> Result<(Self::ReducedStatement, Self::VerifierOutputs), VerifierError>;
}
```

33/33 tests green. `Statement`/`ReducedStatement` shared; oracle types role-split.

Per-phase mapping

	Stmt	Wit	Pr. In	Reduced	Pr. Out
`Pesat`	$(l_1, \log m)$	wits	—	(μ, τ)	cw + tree
`TwinC.`	acc + μ/τ	acc-w + ins	cw + acc-cw	$\gamma, \zeta_0, \beta_\tau$, defer	O[f] + z
`Ood`	$(s, \log n)$	—	O[f]	samples + ans	—
`Batching`	ζ + s, t, log n	—	O[f]	α	μ
`Proximity`	queries + l_2, t	—	trees + cw	()	paths + ans

Asymmetries are now structural:

- `Pesat` — empty `ProverInputs` (it's the source).
- `Proximity` — empty `Reduced` (it's a check).
- `Ood` / `Batching` — empty `Witness`.

Single-protocol view: a candid review

What's good.

- Paper-aligned types — the `IOR` trait and the paper's IOR composition rule line up directly.
- Real verifier symmetry — `TwinConstraint::verify` / `Batching::verify` are first-class trait impls.
- Honest oracle role split — prover sees data, verifier sees commitments.
- Empty-type asymmetries are visible (`Reduced = ()` for checks).
- 33/33 tests green throughout.

What's not.

- Seven associated types per phase — real cognitive load.
- Lifetimes & `PhantomData` proliferate; structs carry `<'a, F, ...>` for anchoring.
- No useful composition combinators — see next slide.
- No type-level ordering enforcement; transcript order is implicit.
- Orchestrator got longer, not shorter — ∼10 lines per phase invocation.

Composition — what the trait can't do

```
pub struct Sequence<A: IOR, B: IOR>(A, B);
impl<A: IOR, B: IOR> IOR for Sequence<A, B>
where
    B::Statement: From<A::ReducedStatement>,
    B::ProverInputs: From<A::ProverOutputs>,
    // ... and the witness has to come from somewhere ...
{ /* run A, derive B's inputs, run B */ }
```

Why no useful generics fall out:

- `From` impls are protocol-specific — written per `(A, B)` pair, no leverage.
- "Between-phases" glue is itself protocol logic (WARP between TC and OOD: compute η, ν_o, build new Merkle tree, write transcript). Doesn't fit either phase's signature.
- Witnesses don't thread cleanly across the boundary.

What unlocks it: a canonical `Claim<F>` algebra of shared shapes — next two slides.

Claim¡F¿ — the shared vocabulary

Lift “what’s being claimed” from per-protocol bags into a small shared enum.

```
pub enum Claim<F: Field> {
    Sumcheck(SumcheckClaim<F>),    // p sums to v on {0,1}^n
    Eval(EvalClaim<F>),            // f_hat(zeta) = y
    Proximity(ProximityClaim<F>),  // codeword delta-close to code
    ZeroCheck(ZeroCheckClaim<F>),  // p == 0 on {0,1}^n
    Batched(Box<BatchedClaim<F>>),
}
```

Operations on these claims become *protocol-agnostic*:

```
pub fn reduce_sumcheck<F, T: Transcript>(
    claim: SumcheckClaim<F>,
    prover: &mut impl SumcheckProver<F>,
    transcript: &mut T,
) -> EvalClaim<F> { ... }
```

STIR uses it. WHIR uses it. WARP uses it. The *transformation* is the abstraction — not the protocol.

Protocol becomes a script over claim transformations

```
fn warp_round<F>(...) -> Claim<F> {
    let c = Claim::ZeroCheck(...);          // R1CS satisfaction
    let c = c.bundle_with(...);             // -> SumcheckClaim
    let c = reduce_sumcheck(c, ...);        // -> EvalClaim at gamma
    let oods = ood_sample(...);             // s EvalClaims
    let qs   = shift_query_claims(...);     // t more
    let c = batch_eval_claims(vec![...]);   // -> 1 EvalClaim
    let c = reduce_sumcheck(c, ...);        // -> EvalClaim at alpha
    Claim::Eval(c)
}
```

Each line is a typed claim transformation. Protocol identity is the *script*; the transformations are reusable.

Limits + cost. Only the public claim flow gets typified — witnesses, transcript threading, setup, and proof shapes stay per-protocol. Canonical shapes only emerge from 3+ implementations, and soundness arguments must thread through the algebra. Multi-protocol research thrust, not a tactical project.

Reframing: a family of protocols

The single-protocol view above was honest — but the actual goal isn't "a clean WARP."

We design protocols like **STIR**, **WHIR**, **WARP** all the time. They share:

- Encode-and-commit (codeword — Reed-Solomon, or otherwise).
- Sumcheck reductions (different flavours: degree-1, multilinear, fused twin-constraint, . . .).
- Out-of-domain (OOD) sampling for proximity boost.
- Shift-query opening with auth paths.
- Batching sumchecks that fold many claims into one.
- Accumulator updates / final consistency.

The bar for the IOR trait then shifts: not "does this clean up WARP?" but "is this the right backbone for a family?"

The toolkit, audited

What practitioners writing STIR / WHIR / WARP *already* have:

- **Transcripts.** `spongefish` 0.7 — de facto in this corner of the ecosystem. Practitioners use it directly. No abstraction needed.
- **Sumchecks.** `effsc` — `SumcheckProver`, `RoundPolyEvaluator`, `InnerProductProver`, `CoefficientProverLSB`, `MultilinearProver`, hypercube + protogalaxy utilities. WARP uses 9 of 10 effsc primitives already.
- **Linear codes.** `ark-codes::LinearCode` — encode, code_len.
- **Vector commitments.** `ark-vc` (abstract trait) + `ark-mt` (Merkle backend, multi-vector + single-shot openings, Blake3 / Poseidon hashers).
- **Oracle struct.** `Vec<F>` codeword + lazy MLE cache. Same storage shape across STIR / WHIR / WARP.

The primitive layer is mostly done. The discussion is really about *integration* — how primitives compose, and what shape protocol authors copy.

What practitioner experience actually needs

Designing a STIR / WHIR / WARP-style protocol is hard. The friction is rarely "I need a new sumcheck primitive." It's:

- **Tying primitives together correctly.** LinearCode + Merkle is two crates; integration is per-protocol boilerplate.
- **Knowing the right phase decomposition.** What's a phase? What's its statement? Where's the witness? When does the verifier act?
- **Threading transcript ordering without bugs.** Squeeze before write, derive challenges in the right segments.
- **Maintaining paper-code alignment.** The paper's IOR composition is the contract; reviewers expect to see it in code.
- **Debugging when soundness fails.** "Where in this 600-line `verify` did the rejection happen?"

The fellow: *practitioners benefit from patterns + examples + shared primitives*, not from cross-protocol trait contracts. That's the lever.

What WARP migrated onto: `ark-vc` / `ark-mt`

The migration replaced `ark-crypto-primitives::merkle_tree` with a multi-vector commitment scheme. Two structural changes:

1. Per-codeword → one tree. PESAT commits l_1 fresh codewords. Leaves are the column-tuples $(c_0[i], \ldots, c_{l_1-1}[i])$, hashed as `Vec<F>`. One root authenticates all codewords; opening returns all m values per index in one call.

2. Per-query → one proof. Old: t queries meant t separate `Path<MT>`, each carrying its own $\log n$ siblings. New: a single `OpeningProof` covering all t positions, with shared upper-layer siblings sent once (path-pruned per book §20: union of copaths minus union of paths).

Concretely, $n = 2^{15}$, $t = 100$:

- Old: $100 \times 15 = 1500$ sibling hashes. Many duplicated near the root.
- New: $\approx 100 \times \log(n/t) \approx 730$ hashes. Savings grow with t.

In WARP types: `MerkleTree<MT>` → `MultiVectorCommitted<H,S,F>`; `Path<MT>` → `OpeningProof<HashRegion<H>>`; `H: MerkleHasher<Symbol = Vec<F>>` replaces `MT: Config` across all phases. Same soundness, less data on the wire, fewer hash invocations on both sides.

What we're for, positively

Keep `IOR` as a structural template, not a cross-protocol contract.

- It's the shape WARP-style protocols copy. The paper section on accumulator-as-state makes it legible.
- Other protocols follow as a *pattern*, get shared mental model + idioms, without being forced into one trait.

Lean on the existing primitive layer.

- `effsc`, `spongefish`, `ark-codes`, `ark-vc` + `ark-mt`.
- Already strong. The discipline is to use them, not reinvent.

WARP runs on `ark-vc`. The missing integration trait was filled by `ark-vc/ark-mt`; WARP migrated onto its multi-vector commitment scheme. STIR-lite plugs into the same surface.

Make WARP the reference implementation. A new protocol designer reads `src/protocol/phases/`, copies the IOR-shaped pattern, plugs in their primitives.

Document the pattern. Short "how to design a Warp-style protocol" guide — narrative, not a trait. The actual lever for practitioner ergonomics.

Where this leaves us — the next move

Paper section (accumulator as typed state). Describe the IOR trait as a paper-aligned realisation *of WARP's protocol structure*, framed as a *template other protocols can follow*, not a contract they must.

Paper section (structured sumcheck primitive). The fused $\alpha/\beta/\tau$-fold inside TwinConstraint is the natural extraction — lives in `effsc` as a new `RoundPolyEvaluator`-shaped impl, citable from outside Warp.

`ark-vc` **integration landed.** WARP migrated from `ark-crypto-primitives::merkle_tree` onto `ark-vc/ark-mt`. Multi-vector openings collapse PESAT's l_1 paths to one proof per query batch. Tests + benches green.

Implement a second protocol. STIR-lite or WHIR-lite, against the same primitives + IOR-shaped pattern. The friction it surfaces is the most reliable guide for the next iteration.

Write the "how to design a Warp-style protocol" guide. Walk through phase decomposition, primitive choices, IOR shape. Narrative document in `docs/`. The actual practitioner-facing artifact.